Universidade Federal de Sergipe (UFS)

Sistemas Operacionais — 2026.2

Relatório de Análise Experimental Estruturada: GenResearch-Servidor

Data da Análise: 16 e 17 de Outubro de 2026

1. Tabela de Contribuição Individual

Abaixo está a tabela de identificação e contribuição dos membros da equipe envolvidos na execução dos testes e na confecção deste relatório analítico.

Nome do Integrante	Matrícula	Contribuição Principal
Beathriz Laurent Carlos Muniz	201900050850	Execução de Testes
Rafael de Jesus Santos	202300114489	Execução de Testes
Luan Feitosa Lima Sátiro	202300061714	Reprodução & automatização de testes
Matheus de Oliveira Lima	202100012062	Criação do relatório
Samyr Teles Sales Silva	202300061830	Criação da apresentação
Evelyn Hilthegarden Santana de Jesus	202300061572	Complemento e formatação de relatório

2. Introdução e Problema

Este relatório técnico expandido descreve minuciosamente a análise dos resultados experimentais obtidos na Avaliação 2 (AV2) do sistema simulado GenResearch-Servidor [cite: 1, 5]. Este ambiente foi desenvolvido para investigar conceitos fundamentais e críticos de sistemas operacionais — como o gerenciamento de processos, escalonamento, uso eficiente de memória, e operações de entrada e saída (E/S) [cite: 1]. Tais conceitos são aplicados a cargas de trabalho intensivas que emulam as encontradas em aplicações de Inteligência Artificial (IA), com foco particular nos modelos de Geração Aumentada por Recuperação (RAG) [cite: 1, 3].

O problema central investigado neste estudo é a identificação e mitigação de gargalos computacionais impostos pelo sistema operacional [cite: 1, 3]. Durante a ingestão de grandes bases de dados, a leitura de modelos vetoriais volumosos e a necessidade de responder a múltiplas requisições simultâneas, o sistema enfrenta desafios substanciais de E/S, paginação e latência [cite: 1, 3].

O estudo aborda o problema comparando abordagens convencionais de leitura de arquivos em oposição ao mapeamento direto de memória (mmap), testando a escalabilidade de filas concorrentes, e demonstrando os riscos de condições de corrida em caches não sincronizados [cite: 1, 3, 4].

3. Pergunta Norteadora

Para guiar a avaliação dos gargalos e do comportamento do sistema, esta análise responde de maneira sistemática e embasada em dados estruturados as seguintes dez perguntas norteadoras [cite: 1]:

- Como tamanho de corpus, índice, lote ou contexto afetou RAM, paginação e latência?
- Que diferença foi observada entre leitura convencional e mmap?
- O cache reduziu tempo ou operações de recuperação? Qual foi a taxa de acerto?
- A concorrência aumentou vazão? Em que ponto gerou contenção, fila ou degradação?
- Que recurso compartilhado exigiu exclusão mútua ou sincronização de eventos?
- Que condição de corrida, risco de deadlock, starvation ou falha de consistência foi demonstrada?
- Qual mecanismo de sincronização foi adotado e quais trade-offs introduziu?
- Houve evidência de paginação intensa, swap, faltas de página ou pressão de memória?
- Como os resultados se relacionam a RAG, embeddings, checkpoints e caches?
- Quais limitações impedem generalizar os resultados para modelos maiores, múltiplas GPUs ou ambientes distribuídos?

4. Ambiente

O ambiente computacional que sustentou as coletas e validações foi operado em plataforma Linux executada como máquina virtual sobre host Windows através do Windows Subsystem for Linux (WSL2) [cite: 2, 4]. As evidências datadas de 17 de Outubro de 2026 descrevem a utilização do Ubuntu 24.04 LTS rodando versões variadas do kernel, notadamente o 6.6.87.2-microsoft-standard-WSL2 e 6.6.114.1 [cite: 2, 3, 4]. O inventário registrou o uso de processadores AMD Ryzen (modelos Ryzen 5 5600GT, Ryzen 5 5500 e Ryzen 7 7435HS) e memória RAM reportada que flutuou entre cerca de 7,7 GiB e 15,51 GiB totais nas diferentes capturas de ambiente [cite: 1, 2, 3, 4]. Embora o sistema dispusesse de placa de vídeo NVIDIA (RTX 2050 ou 3050), não houve registro de uso da mesma para aceleração de cálculo no escopo do servidor simulado [cite: 1, 4].

5. Arquitetura

A arquitetura do GenResearch-Servidor fundamenta-se num servidor HTTP implementado em Python (versão 3.12 documentada nas evidências) [cite: 1, 4]. Esta arquitetura divide a responsabilidade em produtores de requisições web que enfileiram trabalhos, e instâncias de trabalhadores (threads consumidoras) que atuam sobre essas filas descarregando-as [cite: 1, 4]. Para sustentar o fluxo de dados RAG simulado, o sistema possui acesso a um armazenamento persistente e também conta com componentes locais em memória volátil operando como Cache Least Recently Used (LRU) [cite: 1, 4].

6. Dados, Corpus, Embeddings, Índice e Cache

A natureza dos dados manipulados pelo sistema consistiu inteiramente em textos sintéticos criados especificamente para a emulação de carga [cite: 1, 3]. Arquivos de corpus na forma de binários ou textos de extensão ".txt" foram gerados em diferentes testes pesando desde parcos 256 KiB até volumes superiores a 16 MiB [cite: 1, 3].

Diferente de sistemas de Inteligência Artificial genuínos, os vetores e as respostas não provieram de Redes Neurais baseadas em Transformers (como Ollama) ou processamento semântico validado [cite: 1, 4]. Ao invés disso, o sistema simulou os "embeddings" com a aplicação do algoritmo criptográfico de espalhamento PBKDF2 [cite: 1, 4]. Esta escolha de algoritmo exigiu elevado custo de ciclos da CPU em cada requisição, emulando assim a barreira de latência imposta por processos computacionalmente densos [cite: 4]. O índice empregou persistência com a tecnologia de banco de dados SQLite para catalogar os trechos gerados, enquanto o cache temporal manteve estado no espaço de memória local (com capacidade limitada a 8 posições ativas), armazenando o trabalho do gerador PB4KDF2 e viabilizando a economia em execuções redundantes [cite: 1, 4].

7. Processos, Threads, Filas e Recursos Compartilhados

O servidor encapsulou todo o cenário num processo-pai monolítico originado no binário do Python [cite: 1, 4]. Para injetar e consumir paralelismo e testar o escalonamento do Sistema Operacional anfitrião (WSL2 Linux), o executor gerou "trabalhadores" (operando como threads) para varrer solicitações submetidas de forma concorrente em uma fila de requisições capacitada também com o limite de 8 mensagens retidas simultaneamente [cite: 1, 4].

Na dimensão de infraestrutura do SO, a manipulação intensiva exigiu interação constante com os descritores de arquivos, memória primária (RAM e Swap) e a estrutura de alocação de páginas (Page Cache do Kernel) em leituras do SQLite e do arquivo corpus [cite: 1, 3]. Internamente à aplicação, o dicionário que materializa o funcionamento do cache de respostas manifestou-se como o recurso compartilhado primário e crítico da topologia [cite: 1, 4].

8. Estratégia de Sincronização

Uma vez que o cache consistia em memória compartilhada entre as múltiplas threads de trabalhadores encarregadas do atendimento simultâneo, fez-se inevitável o ordenamento no acesso [cite: 1, 4]. A estratégia empregada

consistiu em um mecanismo de barreira no experimento de simulação de erro, seguido pela aplicação de um bloqueio de exclusão mútua integral ("Mutex" ou Lock) abrangendo completamente a fase de checagem, eventual geração computacional dispendiosa do valor e inserção da resposta no dicionário do cache [cite: 1, 4].

9. Metodologia

As coletas experimentais observaram baterias exaustivamente estratificadas: Parte A (inventários de contexto e baseline de hardware), Parte B (prova do fluxo funcional com 8 consultas com acerto validado), Parte C (matriz volumosa isolando a E/S massiva com 42 registros oficiais sob 3 repetições nos cenários C1 a C6 para contornar ruídos transientes) e Parte D (sincronização do cenário de corrida em 6 execuções rigorosas de hits de cache) [cite: 1, 4]. Dados primários sobre comportamento da memória foram angariados lendo estatísticas diretas do pseudo-sistema de arquivos "/proc/PID/io" e mapeamento temporal via "strace", capturando o tamanho do conjunto residente de RAM (RSS) e faltas na tabela de páginas [cite: 1, 4].

10. Resultados

Os dados reportados evidenciam de forma clara como o sistema reage perante diferentes tamanhos de arquivo, abordagens de leitura em disco, uso de cache e escalada concorrência de filas [cite: 1].

Tabela 1: Variação Controlada do Corpus (Experimento C5) [cite: 1]

Corpus KiB	Rep.	Índice B	Trechos	Ingestão ms	Lote ms	RSS KiB	VSZ KiB	minflt
256	1	716800	660	49,902	95,160	30240	334888	2601
256	2	716800	660	49,803	96,560	29784	334888	2382
256	3	716800	660	49,788	97,971	29788	334888	2388
1024	1	2797568	2639	200,981	359,459	38836	412652	6226
1024	2	2797568	2639	197,846	390,939	37944	412652	5985
1024	3	2797568	2639	188,051	362,826	38996	339944	6274

Como disposto na Tabela 1, as rodadas que utilizaram corpo documental quatro vezes maior (1.024 KiB) culminaram com um tempo médio de ingestão

substancialmente superior à faixa dos 49 ms atestados na versão compacta, elevando-o a patamares próximos de 200 ms [cite: 1].

Tabela 2: Leitura Convencional (C1) vs mmap (C2) [cite: 1]

Modo	MiB	Rep.	Leitura ms	RSS pico KiB	RSS fim KiB	VSZ KiB	minflt	read_ bytes
C1	2	1	2,076	40672	25388	37516	516	0
C2	2	1	11,217	40672	25296	37516	548	2158592
C1	16	1	30,033	40672	26336	38596	1028	14618624
C2	16	1	10,356	42480	26432	38596	1145	0

Na Tabela 2, um sumário indicando a leitura das primeiras repetições denota a vantagem do `mmap` (C2) em arquivos volumosos de 16 MiB: concluiu-se na faixa dos 10 ms mediante mapeamento, suplantando os ~30 ms da primeira requisição baseada em cópias convencionais via `read_bytes` de disco pesado do processo de abertura [cite: 1].

Tabela 3: Cache Compartilhado e Acesso (Experimento C4) [cite: 1]

Cache	Padrão	Rep.	Hits	Misses	Hit %	Lat. méd. ms	Vazão req/s
0	Rep.	1	0	12	0,00	2,540	380,4
0	Dist.	1	0	12	0,00	2,589	375,9
8	Rep.	1	11	1	91,67	0,918	1015,9
8	Dist.	1	0	12	0,00	2,653	367,4

A presença do cache, detalhada na Tabela 3, é um ponto de otimização massivo, conseguindo alcançar mais de 90% de taxa de sucesso quando submetida a demandas redundantes, despencando os custos computacionais da barreira de 2,6 ms para a vizinhança relâmpago de 0,9 ms, ampliando consideravelmente a vazão agregada [cite: 1].

Tabela 4: Concorrência e Multithreading na Fila (Experimento C3) [cite: 1]

Trab.	Rep.	Vazão req/s	Lat. méd.	p95 ms	Fila máx.	Espera fila ms	Threads

			ms				
1	1	436,2	8,620	9,244	3	5,647	6
2	1	730,4	5,161	6,189	2	1,651	7
4	1	837,4	4,537	7,016	2	0,111	9

Simultaneamente, a adição controlada de multiplicidade nos trabalhadores encarregados da extração da fila reduziu contundentemente a permanência de requisições pendentes na espera, como destacado na Tabela 4. Com quádruplo processamento, o estrangulamento da fila se tornou praticamente irrelevante [cite: 1].

Tabela 5: Sincronização do Mutex (Experimento D) [cite: 1]

Modo	Rep.	Trab.	Gerações	Duplicadas	Hits	Misses	Lat. méd. ms	Espera mutex ms
D1	1	4	4	3	0	4	32,785	0,000
D2	1	4	1	0	3	1	32,865	23,982

A Tabela 5 comprova o funcionamento da exclusão mútua do Lock D2, a qual erradicou as repetições inúteis exibidas em D1 à custa de mais de vinte milissegundos adicionais de bloqueio da thread em espera [cite: 1].

11. Análise de Memória, Armazenamento, Paginação e E/S

O subsistema de leitura de um arquivo grande impõe obrigatoriedades arquiteturais ao kernel [cite: 3]. O comportamento clássico "convencional", dependente das chamadas "open()" e "read()", atua de forma rigorosa solicitando que as páginas do sistema operacional, mantidas no page cache, sejam duplicadas intrinsecamente na área de memória exclusiva (user space) do aplicativo corrente [cite: 3]. Conforme explicitado na Tabela 2 e logs atrelados, essa replicação integral induz latências tangíveis nos arranques frios em arquivos amplos [cite: 1, 3].

Contrapondo-se ao método usual, a exploração do "mmap" confere outra virtude: a colagem abstrata da geometria total do arquivo perante a Memória Virtual disponível (VSZ) [cite: 3, 4]. Assim se explica o porquê de as faltas de página de menor gravidade ("minflt") serem marginalmente mais altas na medição de

arquivos expressivos na modalidade mmap: o kernel necessita processar a falha de ausência pontual para amarrar aquela página específica do modelo de dados sob demanda [cite: 1].

Nesse ínterim, o experimento que simulava carga ampliada com tamanhos de corpus variáveis apontou perfeitamente uma subida do consumo volumétrico de RAM sem culminar em colapso da unidade lógica. O consumo residente da memória (RSS) alçou degraus aproximados de 29 MiB para 38 MiB com os maiores lotes do teste C5, mas em etapa alguma houve ativação da paginação secundária hostil: as "faltas de página maiores" (majflt) nunca diferiram do marcador primário zero [cite: 1, 4]. A reserva estática de SWAP de 4.112 KiB confirmou não ter existido pressão severa que engatasse repaginação persistente ao disco rígido (condição patológica referida como thrashing) [cite: 1].

12. Análise de Concorrência, Cache e Correção

Sistemas interativos atrelados a inferências demoradas exigem preservação e reaproveitamento enérgico dos resultados. O uso de um Cache Compartilhado atuou perfeitamente como barreira protetora contra o retrabalho cego de hashing de strings longas. Ao responder as interações de contexto repetidas que seriam engavetadas como "miss", a habilitação da capacidade 8 LRU fez a margem de hits despontar para 91,67% (11 sucessos atenuados para somente 1 falha necessária) no lote de 12 averiguações [cite: 1, 4]. A consequência temporal direta foi um estreitamento acentuado da latência rumo aos parcos 0,9 ms médios [cite: 1].

Todavia, tal paralelismo provou não ser infalível e revelou fissuras na sua integridade atômica ("condição de corrida"). Com a finalidade de atestar o perigo, o experimento D1 instigou artificialmente, mas validamente perante a literatura, o gargalo das requisições paralelas num bloco solto de "Read-Modify-Write" do cache. Com quatro processos paralelos averiguando o conteúdo em sincronia, a chave figurou vazia perante o olhar temporal de todas simultaneamente. Houve 3 gerações de dados totalmente repetidas por conta da leitura dessincronizada

(com 0 hits aproveitados) [cite: 1, 4]. O corretivo arquitetado perante este impasse consistiu numa barreira de exclusão mútua por travamento total ("Mutex" D2), convertendo a região crítica de acesso e recriação dos vetores numa fileira de via de mão única, blindando os 3 hits precisos, ao custo temporal oneroso de prender a vazão e adicionar robustos 24 ms de espera ociosa às instâncias adjacentes em fila ("trade-off") [cite: 1]. Não houveram anomalias duradouras ou "deadlocks" mortais presenciados nestas amostragens e interrupções documentadas [cite: 1, 4].

13. Relação com Sistemas Operacionais e IA Generativa

As evidências atestadas nas seções anteriores ecoam substancialmente perante o desenvolvimento presente de infraestrutura alocada em pipelines de "Retrieval-Augmented Generation" e processamento de Grandes Modelos de Linguagem ("LLMs") [cite: 1, 4]. Nestes cenários da borda técnica (como redes baseadas no framework PyTorch, formato Tensor, ou Ollama), não é factível recarregar matrizes que superam múltiplos gigabytes para o "User Space" síncrono a cada iteração do bot. A adoção de mmap atende primorosamente este vácuo, projetando grandes blocos de "embeddings" (vetores densos semânticos precalculados e armazenados) unicamente no espaço virtual do SO e deixando para as faltas de página suaves do Linux a otimização atômica do cache de arquivo [cite: 3, 4]. Além do mais, a latência do pipeline gerador sem cache atesta a dor do reprocessamento integral de tensores sem memória volátil ("Context caches") operante de modo atômico e sincronizado nos "engines" atuais [cite: 1, 3, 4].

14. Limitações

Não obstante a precisão do arranjo experimental na exploração comportamental das camadas adjacentes e processos monolíticos do sistema operacional na memória virtual, a arquitetura provida guarda barreiras fundamentais que proíbem inferências absolutas ao se escalonar projetos em escala em nuvem ou corporativa [cite: 1, 4].

Primordialmente, a abstração da pesquisa de proximidade e carga da IA foi modelada puramente sobre funções de criptografia criptográfica genérica "PBKDF2", com RAG de devolução pseudo-estática, o que está a uma distância estratosférica das verdadeiras multiplicações vetoriais, fluxos matemáticos intrincados e transações flutuantes processadas maciçamente por tensores em GPUs parrudas da vida real [cite: 1, 4]. Conforme as capturas de "iostat" e rastreamento da parte A expuseram, as GPUs reais da máquina (RTX 2050 / RTX 3050) ficaram integralmente à margem da computação, mantendo as pressões no silício do processador x86 sob envoltório de simulação da Hyper-V via "WSL2" [cite: 1, 4]. Em segundo plano, arquiteturas distribuídas robustas enfrentam barreiras de latência oriundas das interconexões Ethernet/InfiniBand assíncronas que superam ordens de magnitude o Mutex simplificado testado pontualmente local no escopo de dicionários e SQLite [cite: 1, 4].

15. Conclusão

O presente aparato probatório demonstra com nitidez a relação direta de compromisso estabelecida entre a garantia de eficiência no carregamento computacional e a consistência intrínseca dos blocos processados assincronamente [cite: 1, 4]. O teste comprovou a superação provida por mapeamentos virtuais contínuos baseados na demanda em E/S espessa, dispensando cópias paralisantes e maximizando o fluxo elástico da paginação suave do Linux [cite: 1, 3]. Na vertente de escalabilidade, embora as baterias paralelas tenham demonstrado que é exequível estancar tempos perdedores de filas web unicamente via alocação de multiplicidade ("workers"), o problema das transações cruzadas no espaço compartilhado ressaltou de forma imediata. Para salvar sistemas pseudo-RAG da repetição dispendiosa do modelo via exclusão atômica em nível de memória em uso, forjou-se um retardo indesejável, ratificando a célebre máxima e o trade-off fundamental dos Sistemas Operacionais perante a coordenação segura em tempo e espaço computacional finito [cite: 1, 3, 4].

16. Referências

[1] Catalogo_Resultados_AV2_GenResearch.docx

[2] experimento.txt

[3] mmap_experimento.pdf

[4] Catalogo_Arquivos_Workspace.md